

Prompt A — SPA: Gestión de Gabinetes, Profesionales y Voucher Automático

PARTE 1 — Gestión de Gabinetes desde la UI

1a — Ampliar `SpaTab` para incluir configuración

En `spa.tsx`, cambiar:

```
// De:
type SpaTab = "agenda" | "tratamientos" | "insumos";

// A:
type SpaTab = "agenda" | "tratamientos" | "insumos" | "configuracion";
```

1b — Agregar botón de tab “Configuración”

En el grupo de botones de tabs, agregar después del botón de “Insumos”:

```
<Button
  variant={activeTab === "configuracion" ? "default" : "ghost"}
  size="sm"
  onClick={() => setActiveTab("configuracion")}
  data-testid="tab-configuracion"
>
  <Settings className="h-4 w-4 mr-1" /> Configuración
</Button>
```

Agregar `Settings` a los imports de `lucide-react` si no está.

1c — Estado para gestión de gabinetes

Agregar junto a los otros estados del componente:

```
const [cabinDialogOpen, setCabinDialogOpen] = useState(false);
const [editingCabin, setEditingCabin] = useState<SpaCabin | null>(null);
const [cabinName, setCabinName] = useState("");
const [cabinDescription, setCabinDescription] = useState("");
```

1d — Mutations para gabinetes

```
const createCabinMutation = useMutation({
  mutationFn: async (data: { name: string; description: string }) => {
    const res = await fetch("/api/spa/cabins", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ ...data, isActive: "true" }),
    });
    if (!res.ok) throw new Error("Error al crear gabinete");
    return res.json();
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/spa/cabins"] });
    setCabinDialogOpen(false);
    setCabinName("");
    setCabinDescription("");
    setEditingCabin(null);
    toast({ title: "Gabinete guardado" });
  },
});

const updateCabinMutation = useMutation({
  mutationFn: async ({ id, data }: { id: string; data: { name: string; descriptio
    const res = await fetch(`/api/spa/cabins/${id}`, {
      method: "PATCH",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(data),
    });
    if (!res.ok) throw new Error("Error al actualizar gabinete");
    return res.json();
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/spa/cabins"] });
    setCabinDialogOpen(false);
    setCabinName("");
    setCabinDescription("");
    setEditingCabin(null);
    toast({ title: "Gabinete actualizado" });
  },
});
```

1e — Tab de Configuración (contenido)

Agregar después del bloque `{activeTab === "insumos" && (...)}:`

```

{activeTab === "configuracion" && (
  <Card>
    <CardHeader className="py-3 px-4">
      <div className="flex items-center justify-between">
        <CardTitle className="text-base">Gabinetes / Espacios</CardTitle>
        <Button
          size="sm"
          onClick={() => {
            setEditingCabin(null);
            setCabinName("");
            setCabinDescription("");
            setCabinDialogOpen(true);
          }}
          data-testid="button-new-cabin"
        >
          <Plus className="h-4 w-4 mr-1" /> Nuevo Gabinete
        </Button>
      </div>
      <p className="text-xs text-muted-foreground mt-1">
        Cada gabinete aparece como una fila en el planning diario. Podés agregar
      </p>
    </CardHeader>
    <CardContent className="p-0">
      <Table>
        <TableHeader>
          <TableRow>
            <TableHead>Nombre</TableHead>
            <TableHead>Descripción</TableHead>
            <TableHead>Estado</TableHead>
            <TableHead className="w-[100px]">Acciones</TableHead>
          </TableRow>
        </TableHeader>
        <TableBody>
          {cabins.length === 0 ? (
            <TableRow>
              <TableCell colspan={4} className="text-center py-8 text-muted-foreg">
                No hay gabinetes registrados
              </TableCell>
            </TableRow>
          ) : (
            cabins.map((cabin) => (
              <TableRow key={cabin.id} data-testid={`cabin-row-${cabin.id}`}>
                <TableCell className="font-medium">{cabin.name}</TableCell>
                <TableCell className="text-sm text-muted-foreground">{cabin.descr}</TableCell>
                <Badge
                  variant={cabin.isActive === "true" ? "default" : "secondary"}

```

```

        className="text-xs cursor-pointer"
        onClick={() => updateCabinMutation.mutate({
          id: cabin.id,
          data: { name: cabin.name, description: cabin.description ||
        })}
        title="Clic para cambiar estado"
        data-testid={`badge-cabin-status-${cabin.id}`}
      >
        {cabin.isActive === "true" ? "Activo" : "Inactivo"}
      </Badge>
    </TableCell>
    <TableCell>
      <Button
        variant="ghost"
        size="icon"
        className="h-8 w-8"
        onClick={() => {
          setEditingCabin(cabin);
          setCabinName(cabin.name);
          setCabinDescription(cabin.description || "");
          setCabinDialogOpen(true);
        }}
        data-testid={`button-edit-cabin-${cabin.id}`}
      >
        <Pencil className="h-4 w-4" />
      </Button>
    </TableCell>
  </TableRow>
))
)}
</TableBody>
</Table>
</CardContent>
</Card>
)}

```

1f — Dialog de crear/editar gabinete

Agregar al final del JSX del componente, antes del cierre del `return` :

```

<Dialog open={cabinDialogOpen} onOpenChange={(open) => { if (!open) { setCabinDia
  <DialogContent className="max-w-sm">
    <DialogHeader>
      <DialogTitle>{editingCabin ? "Editar Gabinete" : "Nuevo Gabinete"}</DialogT
    </DialogHeader>
    <div className="grid gap-4 py-4">

```

```

<div>
  <Label>Nombre *</Label>
  <Input
    value={cabinName}
    onChange={(e) => setCabinName(e.target.value)}
    placeholder="Ej: Gabinete 1, Piscina, Sauna..."
    data-testid="input-cabin-name"
  />
</div>
<div>
  <Label>Descripción (opcional)</Label>
  <Input
    value={cabinDescription}
    onChange={(e) => setCabinDescription(e.target.value)}
    placeholder="Ej: Sala de masajes principal"
    data-testid="input-cabin-description"
  />
</div>
</div>
<DialogFooter>
  <Button variant="outline" onClick={() => setCabinDialogOpen(false)}>Cancela
  <Button
    onClick={() => {
      if (!cabinName.trim()) return;
      if (editingCabin) {
        updateCabinMutation.mutate({
          id: editingCabin.id,
          data: { name: cabinName.trim(), description: cabinDescription.trim(
            });
        } else {
          createCabinMutation.mutate({ name: cabinName.trim(), description: cab
        }
      }
    }}
    disabled={!cabinName.trim() || createCabinMutation.isPending || updateCab
    data-testid="button-confirm-cabin"
  >
    {(createCabinMutation.isPending || updateCabinMutation.isPending) ? "Guar
  </Button>
</DialogFooter>
</DialogContent>
</Dialog>

```

PARTE 2 — Campo “Profesional” en turnos

2a — Nueva tabla en `shared/schema.ts`

```
export const spaProfessionals = pgTable("spa_professionals", {
  id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
  name: text("name").notNull(),
  lastName: text("last_name"),
  isActive: text("is_active").default("true"),
});

export const insertSpaProfessionalSchema = createInsertSchema(spaProfessionals).o
export type InsertSpaProfessional = z.infer<typeof insertSpaProfessionalSchema>;
export type SpaProfessional = typeof spaProfessionals.$inferSelect;
```

Agregar `professionalId` **a** `spaAppointments` :

```
// En la tabla spaAppointments, agregar campo:
professionalId: varchar("professional_id"),
```

2b — Migración

```
await db.execute(sql`
  CREATE TABLE IF NOT EXISTS spa_professionals (
    id VARCHAR PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT NOT NULL,
    last_name TEXT,
    is_active TEXT DEFAULT 'true'
  )
`);

await db.execute(sql`
  ALTER TABLE spa_appointments ADD COLUMN IF NOT EXISTS professional_id VARCHAR
`);
```

2c — Endpoints para profesionales

```
app.get("/api/spa/professionals", async (req, res) => {
  try {
    const professionals = await db.select().from(spaProfessionals);
    res.json(professionals);
  } catch { res.status(500).json({ error: "Error" }); }
});

app.post("/api/spa/professionals", async (req, res) => {
```

```

    try {
      const [created] = await db.insert(spaProfessionals).values(req.body).returnin
      res.json(created);
    } catch { res.status(500).json({ error: "Error" }); }
  });

app.patch("/api/spa/professionals/:id", async (req, res) => {
  try {
    const [updated] = await db.update(spaProfessionals)
      .set(req.body)
      .where(eq(spaProfessionals.id, req.params.id))
      .returning();
    res.json(updated);
  } catch { res.status(500).json({ error: "Error" }); }
});

```

2d — En `spa.tsx`, agregar query y tipo

```

type SpaProfessional = {
  id: string;
  name: string;
  lastName: string | null;
  isActive: string | null;
};

// Query:
const { data: professionals = [] } = useQuery<SpaProfessional[]>({
  queryKey: ["/api/spa/professionals"],
});

const activeProfessionals = professionals.filter(p => p.isActive === "true");

```

2e — Agregar campo profesional al form schema

```

// En appointmentFormSchema, agregar:
professionalId: z.string().optional(),

// En defaultValues del form, agregar:
professionalId: "",

```

2f — Campo en el formulario de nuevo/editar turno

Agregar después del campo de tratamiento, antes del campo de huésped del hotel:

```

<FormField control={form.control} name="professionalId" render={({ field }) => (
  <FormItem>
    <FormLabel>Profesional (opcional)</FormLabel>
    <Select onValueChange={field.onChange} value={field.value || ""}>
      <FormControl>
        <SelectTrigger data-testid="select-professional">
          <SelectValue placeholder="Sin asignar" />
        </SelectTrigger>
      </FormControl>
      <SelectContent>
        <SelectItem value="">Sin asignar</SelectItem>
        {activeProfessionals.map((p) => (
          <SelectItem key={p.id} value={p.id}>
            {p.name} {p.lastName || ""}
          </SelectItem>
        ))}
      </SelectContent>
    </Select>
    <FormMessage />
  </FormItem>
)} />

```

2g — Mostrar profesional en la celda del planning y en el detalle del turno

En la celda del planning (dentro del div de cada turno en el grid), agregar debajo del horario:

```

{appointment.professionalId && (
  <div className="text-[10px] opacity-60 truncate">
    {professionals.find(p => p.id === appointment.professionalId)?.name || ""}
  </div>
)}

```

En el panel de detalle del turno (donde dice `selectedAppointment`), agregar:

```

<div>
  <h4 className="text-sm font-medium">Profesional</h4>
  <p className="text-sm text-muted-foreground">
    {selectedAppointment.professionalId
      ? professionals.find(p => p.id === selectedAppointment.professionalId)?.name
      : "Sin asignar"}
  </p>
</div>

```


2h — Gestión de profesionales en tab Configuración

En el tab de configuración (debajo de la card de Gabinetes), agregar una segunda card:

```
<Card className="mt-4">
  <CardHeader className="py-3 px-4">
    <div className="flex items-center justify-between">
      <CardTitle className="text-base">Profesionales / Masajistas</CardTitle>
      <Button size="sm" onClick={() => setProfessionalDialogOpen(true)} data-test="
        <Plus className="h-4 w-4 mr-1" /> Nueva Profesional
      </Button>
    </div>
  </CardHeader>
  <CardContent className="p-0">
    <Table>
      <TableHeader>
        <TableRow>
          <TableHead>Nombre</TableHead>
          <TableHead>Estado</TableHead>
          <TableHead className="w-[80px]">Acciones</TableHead>
        </TableRow>
      </TableHeader>
      <TableBody>
        {professionals.length === 0 ? (
          <TableRow>
            <TableCell colspan={3} className="text-center py-6 text-muted-foreground">
              No hay profesionales registradas
            </TableCell>
          </TableRow>
        ) : (
          professionals.map((p) => (
            <TableRow key={p.id} data-testid={`professional-row-${p.id}`}>
              <TableCell className="font-medium">{p.name} {p.lastName || ""}</Tab
              <TableCell>
                <Badge
                  variant={p.isActive === "true" ? "default" : "secondary"}
                  className="text-xs cursor-pointer"
                  onClick={() => updateProfessionalMutation.mutate({ id: p.id, is
                    data-testid={`badge-professional-status-${p.id}`}
                >
                {p.isActive === "true" ? "Activa" : "Inactiva"}
              </Badge>
            </TableCell>
            <TableCell>
              <Button variant="ghost" size="icon" className="h-8 w-8"
                onClick={() => { setEditingProfessional(p); setProfessionalName
                  data-testid={`button-edit-professional-${p.id}`}
                >
```

```

        >
        <Pencil className="h-4 w-4" />
      </Button>
    </TableCell>
  </TableRow>
)
  })
</TableBody>
</Table>
</CardContent>
</Card>

```

Agregar estados y mutations correspondientes:

```

const [professionalDialogOpen, setProfessionalDialogOpen] = useState(false);
const [editingProfessional, setEditingProfessional] = useState<SpaProfessional |
const [professionalName, setProfessionalName] = useState("");
const [professionalLastName, setProfessionalLastName] = useState("");

const createProfessionalMutation = useMutation({
  mutationFn: async (data: { name: string; lastName: string }) => {
    const res = await fetch("/api/spa/professionals", {
      method: "POST", headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ ...data, isActive: "true" }),
    });
    return res.json();
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/spa/professionals"] });
    setProfessionalDialogOpen(false); setProfessionalName(""); setProfessionalLas
    toast({ title: "Profesional guardada" });
  },
});

const updateProfessionalMutation = useMutation({
  mutationFn: async ({ id, ...data }: { id: string; name?: string; lastName?: str
    const res = await fetch(`/api/spa/professionals/${id}`, {
      method: "PATCH", headers: { "Content-Type": "application/json" },
      body: JSON.stringify(data),
    });
    return res.json();
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/spa/professionals"] });
    setProfessionalDialogOpen(false); setProfessionalName(""); setProfessionalLas
    toast({ title: "Profesional actualizada" });
  },
});

```

```
    },  
  });  
};
```

Y el dialog de profesional:

```
<Dialog open={professionalDialogOpen} onOpenChange={(open) => { if (!open) { setP  
  <DialogContent className="max-w-sm">  
    <DialogHeader>  
      <DialogTitle>{editingProfessional ? "Editar Profesional" : "Nueva Profesion  
    </DialogHeader>  
    <div className="grid gap-4 py-4">  
      <div>  
        <Label>Nombre *</Label>  
        <Input value={professionalName} onChange={(e) => setProfessionalName(e.ta  
      </div>  
      <div>  
        <Label>Apellido</Label>  
        <Input value={professionalLastName} onChange={(e) => setProfessionalLastN  
      </div>  
    </div>  
    <DialogFooter>  
      <Button variant="outline" onClick={() => setProfessionalDialogOpen(false)}>  
      <Button  
        onClick={() => {  
          if (!professionalName.trim()) return;  
          if (editingProfessional) {  
            updateProfessionalMutation.mutate({ id: editingProfessional.id, name:  
          } else {  
            createProfessionalMutation.mutate({ name: professionalName.trim(), la  
          }  
        }}  
        disabled={!professionalName.trim() || createProfessionalMutation.isPendin  
        data-testid="button-confirm-professional"  
      >  
        Guardar  
      </Button>  
    </DialogFooter>  
  </DialogContent>  
</Dialog>
```

PARTE 3 — Voucher automático cuando el pago es "Cargo a Habitación"

El problema

Cuando el método de pago es `room_charge`, el consumo se transfiere al folio de la habitación y se liquida en recepción al hacer checkout. Por lo tanto el SPA no debe emitir ningún comprobante fiscal — el comprobante lo emite recepción. El sistema debe autoseleccionar “Voucher” y cerrar el folio sin pedir confirmación del tipo de comprobante.

Fix en `spa.tsx`

Buscar el `addPaymentMutation` y en su `onSuccess`, agregar la lógica de cierre automático:

```
const addPaymentMutation = useMutation({
  // ... mutationFn existente sin cambios ...
  onSuccess: (_, variables) => {
    refetchAccount();
    setIsAddPaymentOpen(false);
    setPaymentAmount("");
    setPaymentMethod("");
    setPaymentReservationId("");
    setIsPaymentAdvance(false);
    toast({ title: "Pago registrado" });

    // Si fue cargo a habitación, cerrar folio automáticamente como voucher
    if (variables.method === "room_charge" && selectedAccount) {
      closeAccountMutation.mutate({
        accountId: selectedAccount.id,
        receiptType: "voucher",
      });
    }
  },
});
```

Fix adicional: ocultar selector de comprobante cuando el saldo ya fue cargado a habitación

En el bloque de cierre del folio, verificar si hay un pago `room_charge` registrado y si es así mostrar un mensaje en lugar del selector:

```
{selectedAccount.status === "open" && (
  <div className="border-t pt-4 space-y-3">
    {selectedAccount.payments?.some(p => p.method === "room_charge")} ? (
      <div className="text-sm text-center text-muted-foreground bg-muted/50 rounded
        <Receipt className="h-4 w-4 mx-auto mb-1 text-blue-500" />
```

```

        El consumo fue cargado a la habitación.<br />
        <span className="text-xs">Se emitirá Voucher automáticamente al cerrar.</
    </div>
  ) : (
    <div>
      <label className="text-sm font-medium">Comprobante</label>
      <Select value={receiptType} onChange={setReceiptType}>
        { /* ... opciones existentes sin cambios ... */ }
      </Select>
    </div>
  )}
  <Button
    className="w-full"
    disabled={accountBalance > 0 || closeAccountMutation.isPending ||
      (!selectedAccount.payments?.some(p => p.method === "room_charge") && !rec
    onClick={() => closeAccountMutation.mutate({
      accountId: selectedAccount.id,
      receiptType: selectedAccount.payments?.some(p => p.method === "room_charg
    })}
    data-testid="button-close-folio"
  >
    {closeAccountMutation.isPending && <Loader2 className="h-4 w-4 mr-2 animate
  <Receipt className="h-4 w-4 mr-2" />
    Cerrar Folio
  </Button>
  {accountBalance > 0 && (
    <p className="text-xs text-red-500 text-center">Saldo pendiente: ${accountB
  )}
  </div>
)}

```

Lo mismo para Restaurant

En el módulo de restaurante, buscar el mismo patrón de selector de comprobante al cerrar la cuenta de una mesa. Aplicar el mismo fix: si el método de pago registrado es `room_charge`, autoseleccionar `"voucher"` como comprobante y no mostrar el selector.

Resumen de archivos modificados

Archivo	Cambio
shared/schema.ts	Nueva tabla <code>spaProfessionals</code> , campo <code>professionalId</code> en <code>spaAppointments</code>

<code>server/migrate.ts</code>	CREATE TABLE spa_professionals + ALTER TABLE spa_appointments
<code>server/routes.ts</code>	3 endpoints nuevos para profesionales
<code>client/src/pages/spa.tsx</code>	Tab Configuración, gestión de gabinetes UI, campo profesional en form, voucher automático